

gemstone-rs Cookbook

Task-focused Rust recipes for GemStone/S



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

Cookbook

- Recipe 1: Open a Session and Evaluate Smalltalk
- Recipe 2: Verify ``3 + 4``
- Recipe 3: Write and Read ``UserGlobals``
- Recipe 4: Commit a Write Safely
- Recipe 5: Abort on Error
- Recipe 6: Convert Rust Values to OOPs
- Recipe 7: Fetch a String
- Recipe 8: Send a Message and Keep the Raw OOP
- Recipe 9: Send a Message and Marshal the Result
- Recipe 10: Retain a Long-Lived OOP
- Recipe 10A: Store a Rust Payload Under BridgeRoot
- Recipe 11: Browse Dictionaries
- Recipe 12: Browse a Class
- Recipe 13: Diagnose a New Machine
- Recipe 14: Inspect an OOP From the CLI
- Recipe 15: Inspect BridgeRoot From the CLI
- Recipe 16: Preview Codegen
- Recipe 17: Check Generated Code in CI
- Recipe 18: Generate Wrappers
- Recipe 19: Discover a Config From a Live Stone
- Recipe 20: Run the Local Explorer
- Recipe 19: Enable Explorer Eval Explicitly
- Recipe 20: Use the VS Code Workbench With a Checkout
- Recipe 21: Verify Published Artifacts

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env()??);
println!("{:?}", session.eval("3 factorial")?);
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify $3 + 4$

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()??);
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read UserGlobals

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env()??);
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust");
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {  
    let value = session.new_string("committed");  
    session.global_put("GemStoneRsCommitted", value)  
});
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;  
  
let result: gemstone_rs::Result<()> = session.transaction(|session| {  
    let value = session.new_string("will abort");  
    session.global_put("GemStoneRsAborted", value);  
    Err(Error::IllegalOop {  
        operation: "intentional abort",  
    })  
});  
  
assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;  
  
let seven = session.value_to_oop(&Value::SmallInt(7));  
let yes = session.bool_oop(true);  
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello");  
let text = session.fetch_string(string_oop);  
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("retained OOP {}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env())?;
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
]);

let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

The default root is `UserGlobals` at: `#GemStoneRsBridgeRoot`.

Recipe 11: Browse Dictionaries

```
use gemstone_rs::browser::Browser;

let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{}", dictionary);
}
```

Recipe 12: Browse a Class

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Diagnose a New Machine

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
```

The first command checks environment and GCI loading. The second command logs in and verifies that `3 + 4` returns `7`. The JSON form is useful in CI or editor tooling.

Recipe 14: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 15: Inspect BridgeRoot From the CLI

```
gemstone-rs bridge root
gemstone-rs bridge keys
```

```
gemstone-rs bridge get BookingDraft --symbol  
gemstone-rs bridge inspect BookingDraft --symbol  
gemstone-rs bridge put WorkbenchDraft "hello from Rust" --type String  
gemstone-rs bridge remove WorkbenchDraft
```

Use this when object-mapping examples or generated mappings write payloads under `GemStoneRsBridgeRoot`. `bridge keys` is the quickest way to see what is currently stored before inspecting a specific payload. `bridge put` and `bridge remove` are useful live-write smoke tests because they commit one explicit BridgeRoot change at a time.

Recipe 16: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 17: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 18: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 19: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 20: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 19: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval  
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 20: Use the VS Code Workbench With a Checkout

```
{  
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",  
  "gemstoneRs.useCargo": true,  
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"  
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 21: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.0
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun python docs/build_pdf_docs.py.